



Quoth

Sistema de Auto-Aprendizaje para Claude Code

Referencia Técnica Completa

Version 3.1.0 — Abril 2026
Configuracion actual del nodo Montino
Agustin — OpenClaw Infrastructure

Contenido

1. [Vision General](#)
2. [Arquitectura](#)
3. [Sistema de Hooks](#)
4. [Motor de Inteligencia](#)
5. [Daemon de Procesamiento](#)
6. [MCP Tools \(22 herramientas\)](#)
7. [Plugin de Claude Code](#)
8. [Captura de Trayectorias](#)
9. [Sistema de Patrones](#)
10. [Routing de Tareas](#)
11. [Configuracion Actual \(Montino\)](#)
12. [Instalacion y Setup](#)
13. [Flujo de Datos Completo](#)
14. [Roadmap](#)

1. Vision General

Quoth es un sistema de auto-aprendizaje autonomo para Claude Code que captura trayectorias de trabajo, las procesa en patrones reutilizables, y los inyecta en sesiones futuras para mejorar la calidad de las respuestas del agente.

Principio fundamental: Cada sesion de Claude Code genera conocimiento implicito. Quoth lo hace explicito, lo evalua, y lo reutiliza automaticamente.

Componentes principales

Componente	Funcion	Ubicacion
Hooks	Interceptan eventos de Claude Code (edits, prompts, sesiones)	<code>~/.quoth/hooks/</code> (symlinks)
MCP Server	22 herramientas via JSON-RPC 2.0 sobre stdio	<code>quoth-plugin/mcp/</code>
Daemon	Procesa trayectorias en background (JUDGE → DISTILL → CONSOLIDATE)	<code>quoth-plugin/daemon/</code>
Intelligence Engine	PageRank + trigrams para ranking y routing	<code>quoth-plugin/mcp/lib/</code>
SQLite DB	Almacen de patrones, agentes, eventos	<code>~/.quoth/memory.db</code>
Plugin	Empaquetado para Claude Code (<code>.claude-plugin/</code>)	<code>quoth-plugin/.claude-plugin/</code>

2. Arquitectura

Arquitectura modular

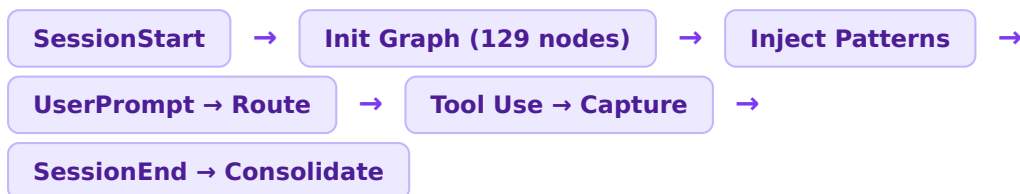
El servidor MCP fue refactorizado de un monolito de 553 lineas a una arquitectura modular:

```

quoth-plugin/
  mcp/
    quoth-learning-server.js    ~55 lines  (protocolo JSON-RPC)
    handlers/
      index.js                 dispatch central
      patterns.js              8 tools   (log, score, search, promote...)
      agents.js                6 tools   (register, heartbeat, assign...)
      intelligence.js           6 tools   (route, init, context, consolidate...)
      skills.js                2 tools   (extract, list)
    lib/
      graph.js                 PageRank, trigrams, Jaccard
      routing.js               TASK_PATTERNS + AGENT_CAPABILITIES
  daemon/
    daemon.js                  Background processor
    db.js                       SQLite layer (better-sqlite3)
  hooks/
    hook-dispatch.js           Unified dispatcher
    inject-patterns.js         Session start pattern injection
    trajectory-capture.js      PostToolUse trajectory logging
    rl-annotate.js             RL signals on MCP tool outputs
  scripts/
    setup.sh                   Automated installation
    .claude-plugin/            Claude Code plugin packaging

```

Flujo de ejecucion por sesion



3. Sistema de Hooks

Los hooks se configuran en `~/.claude/settings.json` y se ejecutan como procesos Node.js independientes. Todos los archivos en `~/.quoth/hooks/` son symlinks al source tree para portabilidad.

Hooks activos

Evento	Handler	Timeout	Funcion
SessionStart	hook-dispatch.js session-restore	15s	Inicializa grafo de inteligencia, inyecta patrones top al contexto
UserPromptSubmit	hook-dispatch.js route	10s	Analiza prompt, sugiere agente optimo, muestra alternativas
PreToolUse (Bash)	hook-dispatch.js pre-bash	5s	Valida comandos peligrosos (rm -rf /, fork bombs, etc.)
PostToolUse (Write Edit)	hook-dispatch.js post-edit	10s	Registra edicion en pending-insights.jsonl
PostToolUse (Bash Write Edit Agent)	trajectory-capture.js	3s	Captura trayectoria completa (tool + input + output)
SessionEnd	hook-dispatch.js session-end	10s	Consolida grafo: decay, refresh, rebuild PageRank
PreCompact (manual auto)	hook-dispatch.js session-end	5-6s	Consolida antes de comprimir contexto
SubagentStop	hook-dispatch.js post-task	5s	Feedback implicito de exito al completar subagente
Stop	auto-memory-hook.mjs sync	10s	Sincroniza memorias (separado de quoth)

Resolucion de paths (portabilidad)

Los hooks usan `fs.realpathSync(__dirname)` para resolver symlinks. Cuando se invoca `~/.quoth/hooks/hook-dispatch.js` (symlink), Node.js resuelve `__dirname` al directorio real del source tree, permitiendo que `path.join(__dirname, '..', 'mcp', 'handlers')` encuentre los modulos correctamente.

Variables de entorno opcionales para override: `QUOTH_HOME` , `QUOTH_PLUGIN_DIR` , `QUOTH_DB_MODULE`

4. Motor de Inteligencia

El motor de inteligencia combina PageRank con similitud de trigramas para ranquear y recuperar conocimiento relevante. Portado de `intelligence.cjs` (ruflo) a módulos puros de Quoth.

Componentes

graph.js — Algoritmos

Funcion	Descripcion
<code>tokenize(text)</code>	Normaliza texto a tokens lowercase
<code>trigrams(tokens)</code>	Genera set de trigramas de tokens consecutivos
<code>jaccardSimilarity(a, b)</code>	Similitud Jaccard entre dos sets de trigramas (0-1)
<code>computePageRank(nodes, edges, opts)</code>	PageRank iterativo con damping=0.85, 20 iteraciones
<code>buildEdges(entries, threshold)</code>	Construye aristas temporales + de similitud (Jaccard > threshold)

Estado persistido

Almacenado en `~/.quoth/intelligence/` :

Archivo	Contenido
<code>graph-state.json</code>	Nodos y aristas del grafo (129 nodos, 95 aristas actualmente)
<code>ranked-context.json</code>	Entradas ranqueadas por PageRank
<code>store.json</code>	Almacen de entradas con metadata
<code>snapshots.json</code>	Snapshots de estado para tracking temporal
<code>last-matched.json</code>	Ultimos matches para RL feedback
<code>pending-insights.jsonl</code>	Ediciones pendientes de procesar

Estadísticas actuales del grafo

Nodos	129	Aristas	95 (87 temporales, 8 similitud)
Densidad	0.0115	Snapshots	2
Confianza min/max/media	0.50 / 0.55 / 0.502		
Top PageRank	0.0756 (Orden de ejecucion sugerido)		

Ciclo de vida del grafo

1. **initGraph(db)** — Lee MEMORY.md files + patrones de la DB, construye nodos, calcula aristas (Jaccard > 0.15), ejecuta PageRank
2. **getContext(prompt, topK)** — Genera trigramas del prompt, calcula Jaccard contra todos los nodos, retorna top-K con scores
3. **consolidateGraph(db)** — Procesa pending edits, refresca desde patterns DB, aplica decay a entradas no accedidas, rebuild completo
4. **applyFeedback(success)** — Ajusta confianza de los ultimos nodos matched (+0.03 exito / -0.03 fallo)

5. Daemon de Procesamiento

El daemon corre en background y procesa trayectorias capturadas usando subagentes de Claude (Haiku) para extraer patrones reutilizables.

Pipeline de procesamiento

1. JUDGE → 2. DISTILL → 3. CONSOLIDATE

Fase	Modelo	Input	Output
JUDGE	Haiku	Trayectoria raw (tool calls + results)	Veredicto: es patron reutilizable? (si/no + razon)
DISTILL	Haiku	Trayectorias aprobadas	Patron destilado: nombre, accion, tags, confianza inicial
CONSOLIDATE	Local	Patrones destilados + existentes	Merge con DB: dedup, update confianza, prune baja confianza

Configuracion del daemon

Parametro	Valor	Descripcion
Auto-start	Via hook <code>session-start</code>	Se inicia automaticamente al abrir sesion
PID file	<code>~/.quoth/daemon.pid</code>	Tracking de proceso
Log file	<code>~/.quoth/daemon.log</code>	Log de actividad
Debug mode	<code>QUOTH_DEBUG=true</code>	Verbose logging

Scoring Bayesiano

La confianza de patrones usa distribucion Beta(alpha, beta) en lugar de incrementos simples:

- **Exito:** $\alpha += 1$ (incrementa media)
- **Fallo:** $\beta += 1$ (decrementa media)
- **Confianza** = $\alpha / (\alpha + \beta)$ — converge con mas evidencia

- **Delta por feedback:** +/-0.03 por evento

Decision Attribution

El sistema trackea que patron causo cada decision exitosa o fallida, permitiendo refinar la confianza de patrones especificos basado en outcomes reales.

Source Tagging

Cada patron tiene un tag de origen:

Tag	Origen
distilled	Extraido automaticamente de trayectorias via daemon
exolar-seeded	Importado de clusters de fallos de Exolar
healer-learned	Aprendido de correcciones de errores
attributed	Refinado via decision attribution
skill-derived	Extraido de tests exitosos como skills

Promocion nocturna

Patrones con **confianza > 0.8** y **> 10 usos** se auto-promueven a Quoth Cloud (quoth.triquail.dev) a las 3am. Re-promocion solo cuando la confianza mejora > 0.1 desde el ultimo upload. Requiere `QUOTH_API_KEY`.

6. MCP Tools (22 herramientas)

Todas las herramientas se exponen via JSON-RPC 2.0 sobre stdio a traves del servidor `quoth-learning-server.js`. Registrado globalmente en `~/.mcp.json`.

Patterns (8 tools) — `handlers/patterns.js`

Tool	Descripcion	Input clave
<code>quoth_log_outcome</code>	Registra exito/fallo de un patron	pattern_id, success (bool)
<code>quoth_score_pattern</code>	Ajusta manualmente la confianza	pattern_id, delta
<code>quoth_top_patterns</code>	Top-N patrones por confianza	limit, tags[]
<code>quoth_search_patterns</code>	Busqueda por texto/tags	query, tags[]
<code>quoth_project_patterns</code>	Patrones de un proyecto especifico	project, limit
<code>quoth_promote_global</code>	Promueve patron a scope global	pattern_id
<code>quoth_seed_from_exolar</code>	Importa fallos de Exolar como candidatos	cluster_data
<code>quoth_propose_update</code>	Promueve patron a Quoth Cloud manualmente	pattern_id

Agents (6 tools) — `handlers/agents.js`

Tool	Descripcion	Input clave
<code>quoth_daemon_status</code>	Estado del daemon (PID, log, uptime)	(ninguno)
<code>quoth_ingest_trajectory</code>	Ingesta manual de trayectoria	trajectory_data
<code>quoth_agent_register</code>	Registra agente en la DB	name, type, capabilities
<code>quoth_agent_heartbeat</code>	Heartbeat de agente activo	agent_id
<code>quoth_agent_list</code>	Lista agentes registrados	status filter
<code>quoth_assign_task</code>	Asigna tarea a un agente	agent_id, task

Intelligence (6 tools) — `handlers/intelligence.js`

Tool	Descripcion	Input clave
<code>quoth_route_task</code>	Sugiere agente optimo para una tarea	task (string)
<code>quoth_intelligence_init</code>	Inicializa grafo desde DB + MEMORY.md	(ninguno)
<code>quoth_intelligence_context</code>	Recupera contexto relevante para un prompt	prompt, topK
<code>quoth_intelligence_consolidate</code>	Consolida grafo (decay + refresh + rebuild)	(ninguno)
<code>quoth_intelligence_stats</code>	Estadísticas del grafo	json (bool)
<code>quoth_intelligence_feedback</code>	Feedback RL (+/- confianza)	success (bool)

Skills (2 tools) — `handlers/skills.js`

Tool	Descripcion	Input clave
<code>quoth_extract_skill</code>	Extrae skill reutilizable de un test exitoso	test_file, test_name
<code>quoth_list_skills</code>	Lista skills extraidos	(ninguno)

7. Plugin de Claude Code

El plugin empaqueta hooks, MCP server, comandos y agentes en formato `.claude-plugin/` para instalacion zero-config.

Estructura

<code>.claude-plugin/</code>	
<code> plugin.json</code>	Manifiesto (nombre, version, MCP, hooks, config)
<code> hooks/</code>	
<code> hooks.json</code>	Declaracion de hooks (PostToolUse, SessionStart)
<code> commands/</code>	
<code> patterns.md</code>	<code>/quoth:patterns</code> – browse pattern library
<code> learn.md</code>	<code>/quoth:learn</code> – trigger consolidacion manual
<code> agents/</code>	
<code> learner.md</code>	<code>quoth:learner</code> – agente Haiku para review de trayectorias

plugin.json

Campo	Valor
name	quoth
version	3.1.0
MCP server	<code>quoth-learning</code> (stdio, Node.js)
Hooks	3: trajectory-capture, rl-annotate, inject-patterns
Commands	2: <code>/quoth:patterns</code> , <code>/quoth:learn</code>
Agents	1: <code>quoth:learner</code> (Haiku)
userConfig	QUOTH_API_KEY (opcional, sensitive)

RL Annotations (rl-annotate.js)

Hook PostToolUse que corre despues de llamadas a tools MCP de Quoth. Busca patrones relevantes en la DB y los inyecta como `additionalContext` con scores de confianza, creando un loop de reinforcement learning.

8. Captura de Trayectorias

Cada tool call que Claude Code ejecuta se captura como un entry en un archivo JSONL por sesion.

Formato de captura

```
{
  "timestamp": 1712170800000,
  "session": "abc123",
  "tool": "Edit",
  "input": { "file_path": "/src/app.ts", "old_string": "...",
  "new_string": "..."},
  "output_summary": "File updated successfully",
  "project": "quoth"
}
```

Almacenamiento

Parametro	Valor actual
Directorio	~/quoth/trajectories/
Formato	Un .jsonl por sesion de Claude Code
Sesiones capturadas	2 archivos
Entries totales	382 lineas
Tools capturados	Bash, Write, Edit, MultiEdit, Agent

9. Sistema de Patrones

Ciclo de vida de un patron

1. **Captura:** trajectory-capture.js graba tool calls en .jsonl
2. **Evaluacion:** Daemon JUDGE evalua si la trayectoria es patron reutilizable
3. **Destilacion:** Daemon DISTILL extrae nombre, accion, tags
4. **Almacenamiento:** Se inserta en `memory.db` con confianza $\text{Beta}(1,1) = 0.50$
5. **Uso:** inject-patterns.js lo inyecta en sesiones futuras si es top-8
6. **Feedback:** Exito/fallo ajusta confianza via `quoth_log_outcome`

7. **Promocion:** Si confianza > 0.8 y > 10 usos, se promueve a Quoth Cloud

Inyeccion token-optimizada

La inyeccion de patrones al inicio de sesion usa un formato comprimido para minimizar uso de contexto:

```
[Quoth] 8 patterns loaded
HIGH: pattern-auth(0.9), error-handling(0.8)
MED: api-design(0.6), test-setup(0.5)
Details:
- pattern-auth: Use JWT with refresh tokens, validate at middleware level...
- error-handling: Wrap external calls in try/catch, log context...
- api-design: REST endpoints follow /api/v1/{resource} convention...
```

Budget de tokens: Maximo 800 caracteres (~200 tokens). Patrones agrupados por tier (HIGH/MED). Solo los top 3 incluyen texto de accion completo. Reduccion: ~800 tokens (verbose) → ~200 tokens (comprimido). **Reduccion del 75%.**

Estado actual de la DB

Metrica	Valor
Tamano de DB	76 KB
Total patrones	2
Alta confianza (≥ 0.8)	0
Media confianza (0.5-0.8)	2
Baja confianza (< 0.5)	0

10. Routing de Tareas

El routing analiza el prompt del usuario y sugiere el tipo de agente optimo para la tarea. Portado de `router.js` (ruflo).

Metodo: Keyword Matching

`routing.js` define patrones regex para cada tipo de agente:

Agente	Patron	Confianza
tester	test spec coverage unit test integration	80%
reviewer	review audit check inspect quality	80%
security-auditor	security vuln cve owasp pentest	85%
devops	deploy docker ci cd pipeline infrastructure	80%
researcher	research investigate analyze explore study	75%
planner	plan design architect structure organize	75%
coder	(default fallback)	50%

Output del routing

En cada `UserPromptSubmit`, el hook muestra:

```
[INFO] Routing task: fix the authentication bug in middleware

+----- Primary Recommendation -----+
| Agent: coder                               |
| Confidence: 50.0%                         |
| Reason: Default routing - no specific pattern matched |
+-----+

Alternative Agents
+-----+-----+-----+
| Agent Type | Confidence | Reason |
+-----+-----+-----+
| tester     | 60.0%    | Alternative agent |
| reviewer   | 50.0%    | Alternative agent |
+-----+-----+-----+
```


11. Configuracion Actual (Montino)

Estado del sistema a fecha 2026-04-04.

Archivos de configuracion

Archivo	Contenido relevante
<code>~/.claude/settings.json</code>	9 hooks (todos apuntan a <code>\$HOME/.quoth/hooks/</code>), MCP server <code>skill-registry</code> , permisos
<code>~/.claude/settings.local.json</code>	<code>enabledMcpjsonServers</code> : <code>["clerk", "quoth-learning"]</code> , <code>enableAllProjectMcpServers</code> : <code>true</code>
<code>~/.mcp.json</code>	MCP global: <code>quoth-learning</code> (stdio, Node.js)

Estructura de archivos en disco

<code>~/.quoth/</code>		
<code>memory.db</code>	76 KB	SQLite (patterns, agents, events)
<code>daemon.pid</code>		PID del daemon activo
<code>daemon.log</code>		Log de procesamiento
<code>intelligence/</code>		
<code>graph-state.json</code>		Grafo: 129 nodos, 95 aristas
<code>ranked-context.json</code>		Entradas ranqueadas por PageRank
<code>store.json</code>		Almacen de metadata
<code>snapshots.json</code>		2 snapshots
<code>last-matched.json</code>		Ultimos matches para RL
<code>pending-insights.jsonl</code>		Cola de ediciones pendientes
<code>trajectories/</code>		
<code>{session-id}.jsonl</code>		2 archivos, 382 entries totales
<code>hooks/ (symlinks)</code>		
<code>hook-dispatch.js</code>	->	<code>quoth-plugin/hooks/hook-dispatch.js</code>
<code>inject-patterns.js</code>	->	<code>quoth-plugin/hooks/inject-patterns.js</code>
<code>rl-annotate.js</code>	->	<code>quoth-plugin/hooks/rl-annotate.js</code>
<code>trajectory-capture.js</code>	->	<code>quoth-plugin/hooks/trajectory-capture.js</code>

Eliminacion de ruflo

Migrado el 2026-04-03/04. Ruflo (@claude-flow/cli) eliminado completamente:

- 10 procesos zombie matados (382 MB liberados)
- 821 paquetes npm desinstalados
- 17 archivos `.mcp.json` limpiados (6 proyectos + IPS + 10 openclaw workspaces)
- 50+ directorios `.claude-flow/` eliminados (~72 MB)
- 200+ MCP tools de ruflo reemplazados por 22 tools de quoth
- Zero referencias a claude-flow en todo el sistema

12. Instalacion y Setup

Instalacion automatica

```
git clone <quoth-repo>
cd quoth-plugin
bash scripts/setup.sh
```

Que hace setup.sh

1. Crea estructura `~/ .quoth/` (hooks/, intelligence/, trajectories/)
2. Symlink de 4 hook files desde source tree a `~/ .quoth/hooks/`
3. Inyecta hook declarations en `~/ .claude/settings.json` (merge, no overwrite)
4. Agrega permiso `Bash(node .quoth/*)`
5. Muestra instrucciones para iniciar daemon

Idempotente: El script detecta si ya esta instalado y no duplica configuracion. Seguro re-ejecutar.

Verificacion

```
# Verificar hooks
node ~/ .quoth/hooks/hook-dispatch.js stats

# Verificar MCP server (22 tools)
echo '{"jsonrpc":"2.0","id":1,"method":"tools/list"}' | \
  node quoth-plugin/mcp/quoth-learning-server.js

# Iniciar daemon
node quoth-plugin/daemon/daemon.js &
```

13. Flujo de Datos Completo

```

Usuario escribe prompt
|
v
[UserPromptSubmit Hook]
|
+---> hook-dispatch.js route
|
|       +---> intelligence.getContext(prompt, 5)
|       |       lee ranked-context.json
|       |       calcula Jaccard vs trigramas del prompt
|       |       retorna top-5 patterns relevantes
|       |
|       +---> routing.routeTask(prompt)
|       |       regex match contra TASK_PATTERNS
|       |       retorna agente recomendado + alternativas
|
v
Claude Code ejecuta herramienta
|
v
[PostToolUse Hook]
|
+---> trajectory-capture.js
|       lee stdin (tool_name, tool_input, tool_output)
|       append a ~/.quoth/trajectories/{session}.jsonl
|
+---> hook-dispatch.js post-edit (si Write|Edit)
|       append a intelligence/pending-insights.jsonl
|
+---> rl-annotate.js (si mcp__quoth-learning__*)
|       busca patrones relevantes por tag en DB
|       retorna additionalContext con scores
|
v
[SubagentStop Hook]
|
+---> hook-dispatch.js post-task
|       intelligence.applyFeedback(true)
|       +0.03 confianza a ultimos matched patterns
|
v
[SessionEnd / PreCompact Hook]
|
+---> hook-dispatch.js session-end
|       intelligence consolidateGraph(db)
|       procesa pending edits
|       refresca desde patterns DB
|       decay entradas no accedidas
|       rebuild PageRank completo
|
v
[Daemon (background)]
|
+---> Lee trajectories/*.jsonl
+---> JUDGE (Haiku): es patron?

```

```
+---> DISTILL (Haiku): extrae patron
+---> CONSOLIDATE: merge a memory.db
+---> 3am: promueve high-confidence a Cloud
|
v
[Siguiente SessionStart]
|
+---> hook-dispatch.js session-restore
      intelligence.initGraph(db)
      lee memory.db + MEMORY.md files
      construye grafo (129 nodos, 95 aristas)
      inyecta top-8 patterns al contexto (~200 tokens)
```

14. Roadmap

Implementado (v3.1.0)

- Servidor MCP modular (22 tools, 4 handlers)
- Motor de inteligencia (PageRank + trigrams + Jaccard)
- Hook dispatcher unificado
- Captura de trayectorias
- Daemon JUDGE/DISTILL/CONSOLIDATE
- Scoring Bayesiano + Decision Attribution
- Token-optimized pattern injection
- RL annotations (PostToolUse)
- Plugin Claude Code (.claude-plugin/)
- Automated setup (setup.sh)
- Portabilidad (symlinks + realpathSync)
- Promocion nocturna a Quoth Cloud

Pendiente (Fases 4-6)

Fase	Feature	Tools
4	Browser Automation	quoth_browser_open, screenshot, click, fill, eval, close (CDP)
5	Workflow Engine	quoth_workflow_create, run, status, list (kv_store-backed)
6	Terminal Management	quoth_terminal_execute, list, close (child_process)